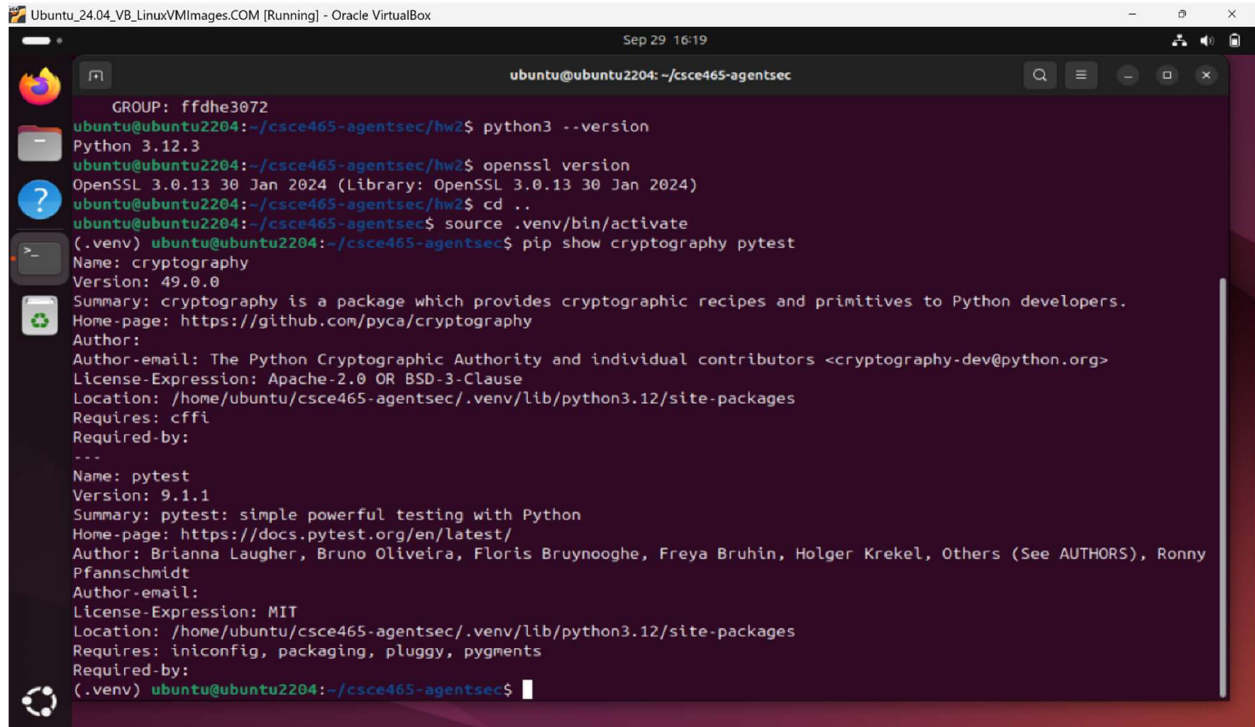


CSCE-465 HW 2: Protect Agent Messages with Classic Cryptography

Matt Mcrory

Github Repo: <https://github.com/mattmccrory/CSCE465-HW2.git>

Task 0



```
Ubuntu_24.04_VB_LinuxVMImages.COM [Running] - Oracle VirtualBox
Sep 29 16:19
ubuntu@ubuntu2204: ~/csce465-agentsec
GROUP: ffdhe3072
ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ cd ..
ubuntu@ubuntu2204:~/csce465-agentsec$ source .venv/bin/activate
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$
```

Task 1

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python baseline_ctr.py
Original plaintext: {"action":"READ","path":"notes.txt"}

Original ciphertext: b90c1d45e6e3c37ca593b6905cc89051321fc6ffb68dbd000a2619fae6305
9f85b05e51c

Modified ciphertext: b90c1d45e6e3c37ca593b6954bc08051321fc6ffb68dbd000a2619fae6305
9f85b05e51c
Modified plaintext: {"action":"WRIT","path":"notes.txt"}

XOR differences:
Position 11: 52 XOR 57 = 05
Position 12: 45 XOR 52 = 17
Position 13: 41 XOR 49 = 08
Position 14: 44 XOR 54 = 10

--- Replay Attack ---
First transmission:
Receiver processed: {"action":"READ","path":"notes.txt"}

Second transmission (replay):
Receiver processed: {"action":"READ","path":"notes.txt"}

Total commands processed: 2
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$
```

I implemented an AES-CTR encryption and a matching program to decrypt the message. As an attacker, I also made a relay function that intercepts and modifies the ciphertext.

During the experiment, I was able to modify the encrypted message without having the key using XOR differences. This was done by replaying the same ciphertext and processing the command twice.

The root cause was AES-CTR not providing integrity or authentication while providing confidentiality. An attacker can predictably modify a message because it uses XOR in the encryption. There is also no authentication to confirm the message was not modified.

Task 2

The handshake.py program a ephemeral Diffie-Hellman key pair using the provided ffdhe3072 file. Each side generates a nonce and then exchanges their public values and nonces. From there they form a shared transcript containing all the data required for both sides to maintain a secure connection. This allows both sides to create the same secret without needing to send it over the network.

This method prevents an attacker from using their own DH values by having both sides of the handshake use their RSA private key to access the transcript. The two sides verify each other's signatures using the RSA public key. Without the signatures an attacker could perform man in the middle attacks.

Task 3

```
pytest -v tests/test_secure_record.py
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 --
/home/ubuntu/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/ubuntu/csce465-agentsec/hw2
collected 6 items

tests/test_secure_record.py::test_valid_bidirectional_communication
PASSED [ 16%]
tests/test_secure_record.py::test_modified_ciphertext_is_rejected
PASSED [ 33%]
tests/test_secure_record.py::test_modified_authenticated_header_is_rejected
PASSED [ 50%]
tests/test_secure_record.py::test_replayed_record_is_rejected
PASSED [ 66%]
tests/test_secure_record.py::test_wrong_direction_record_is_rejected
PASSED [ 83%]
tests/test_secure_record.py::test_invalid_record_does_not_advance_sequence
PASSED [100%]

===== 6 passed in 0.06s =====
```

Task 4

```
pytest -v tests/test_handshake.py
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 --
/home/ubuntu/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/ubuntu/csce465-agentsec/hw2
collected 7 items

tests/test_handshake.py::test_valid_handshake PASSED
[ 14%]
tests/test_handshake.py::test_changed_ciphertext_transcript_nonce
PASSED [ 28%]
tests/test_handshake.py::test_changed_public_value PASSED
[ 42%]
tests/test_handshake.py::test_malformed_transcript PASSED
[ 57%]
tests/test_handshake.py::test_unexpected_identity PASSED
[ 71%]
tests/test_handshake.py::test_invalid_signature PASSED
[ 85%]
tests/test_handshake.py::test_reflected_handshake_message PASSED
[100%]

===== 7 passed in 2.93s =====
```

The other test results can be viewed in the Task 3 section of the report.

Security Note:

The goal of the system is to provide protected messages between two users with an authenticated gateway and a node from attackers. The design was made with the assumption that the attacker has the ability to observe, capture, modify, inject, replay, or just generally interfere with the handshake messages, but does not possess a private key of one of the two users. The design is not protected against a compromise in one of the users. The cryptography gives the message confidentiality, but the cryptography itself does not ensure integrity or authentication.

AES-CTR is a prime example of that idea. It provides confidentiality but not integrity. For CTR to remain secure a different IV has to be used whenever using the same encryption key. Reusing the same IV can result in the same keystream. This gives the attackers an opportunity to XOR the two messages and allow them to recover the information in the encrypted messages. With this in mind, my program ensures the IV is different each time but using a sequence ID that increases with every message. That way an IV cannot be used twice in the same session.

Before decrypting the record, the HMAC must be verified by the receiver. This is an important step in preventing an attacker from modifying the message before it arrives at its destination. Without that crucial verification the receiver would decrypt the message before confirming whether it was untampered. My program checks the structure and HMAC first to properly ensure the integrity of the information. It only decrypts the data after the HMAC is verified.

The system also has an extra layer of both encryption and MAC keys instead of just using one for two different jobs. There is a separate encryption and key for the gateway to node route and another encryption and key for the node to gateway route. This doubling up and separate of keys manages where each encryption is used and what its job is. It prevents one key from being reused for another purpose other than its original one.

A different encryption method we could have used is AES-GCM, which falls under encryptions with a AEAD mode. AES-GCM just makes a single construction with both encryption and authentication. It makes the process a lot more streamlined and straightforward, eliminating some of the required overhead logic of AES-CTR. The design I used (encryption then MAC) explicitly separates integrity and confidentiality but that needs to be handled skillfully with HMAC. Using AES-GCM would still call for managing nonces.

Lastly, once again I would like to point out that this design provides protection assuming that one of the users is not compromised. The encryption and authentication will not stop a malicious tool call sent in an authorized message. If it is sent from an authorized source, then it will be accepted because it was not modified in transit. For the design to be totally and completely secure it would have to have a separate way of determining if a message is safe or holds something malicious. Other safety checks are still required even with a successful encryption.